

ALGORITMOS DE BÚSQUEDA DE PATRONES EN CADENAS

1. Introducción y Definición del Problema

El procesamiento eficiente de cadenas de caracteres constituye uno de los pilares fundamentales dentro de las ciencias de la computación y la ingeniería de software. En la era actual, la manipulación, análisis y recuperación de información textual son tareas críticas que impactan desde los motores de búsqueda web y los sistemas de gestión de bases de datos hasta la bioinformática y la ciberseguridad.

Dentro de este vasto campo, el problema central de la búsqueda de patrones se define de la siguiente manera: dado un texto principal T de longitud n y una secuencia de caracteres más corta denominada patrón P de longitud m (donde $m \leq n$), ambos compuestos sobre un alfabeto finito Σ , el objetivo consiste en identificar y localizar todas las posiciones iniciales o índices dentro del texto T en los cuales el patrón P aparece de forma íntegra y exacta.

Para comprender formalmente este problema, es necesario descomponer sus elementos principales:

- **El Alfabeto (Σ):** Representa el conjunto finito y no vacío de símbolos permitidos que conforman tanto el texto como el patrón (por ejemplo, el conjunto de caracteres ASCII, el alfabeto latino o secuencias binarias).
- **El Texto (T):** Es la secuencia principal de símbolos de longitud n , $T = t_0 t_1 t_2 \dots t_{n-1}$, sobre la cual se efectúa el proceso de exploración y búsqueda.
- **El Patrón (P):** Es la secuencia objetivo de longitud m , $P = p_0 p_1 p_2 \dots p_{m-1}$, que se desea encontrar dentro del texto.
- **El Espacio de Ocurrencias:** El conjunto de todos los índices i (con $0 \leq i \leq n - m$) tales que $T[i \dots i + m - 1] = P[0 \dots m - 1]$.

La relevancia práctica de este problema radica en su ubicuidad. En la biología computacional, los algoritmos de búsqueda de patrones se emplean para alinear secuencias de ADN y ARN, identificando mutaciones o genes específicos. En los sistemas operativos y editores de código, facilitan funciones avanzadas de búsqueda y reemplazo en tiempo real. Asimismo, en los sistemas de detección de intrusos (IDS), permiten inspeccionar flujos de tráfico de red para reconocer firmas de código malicioso de manera inmediata.

Implementación computacional del problema: Para comprobar el funcionamiento de los algoritmos estudiados, se desarrollarán implementaciones en lenguaje C que permitan ingresar un texto y un patrón como datos de entrada. Los programas tendrán como objetivo identificar todas las posiciones en las que el patrón aparece dentro del texto. Para ello, se implementarán tres estrategias de búsqueda: búsqueda ingenua o

fuerza bruta, Knuth-Morris-Pratt (KMP) y Rabin-Karp. De esta manera, será posible comparar experimentalmente sus procedimientos, resultados y características computacionales.

2. Estudio de Algoritmos de Búsqueda de Patrones

Para resolver el problema de la búsqueda de patrones, la literatura informática ha desarrollado diversas estrategias que varían desde enfoques puramente intuitivos hasta métodos altamente sofisticados basados en teoría de números y estructuras de preprocesamiento.

Búsqueda Ingenua (Fuerza Bruta)

El método de búsqueda ingenua representa la aproximación más directa y conceptualmente sencilla para resolver el problema. Su lógica consiste en alinear el inicio del patrón con el primer carácter del texto y proceder a comparar los caracteres uno a uno, avanzando de izquierda a derecha.

Si todos los caracteres coinciden de manera consecutiva, se registra la posición como una ocurrencia válida. Sin embargo, en el momento en que se detecta un desajuste entre un carácter del texto y el patrón, el proceso de comparación actual se interrumpe, el patrón se desplaza exactamente una posición hacia la derecha con respecto al texto, y el proceso de comparación se reinicia desde el principio del patrón. Esta falta de reutilización de la información obtenida en comparaciones previas es la principal causa de su ineficiencia en escenarios extensos.

A continuación, el ejemplo del algoritmo de fuerza bruta al que le di de datos un trabalenguas, el código está en el archivo fuerzabruta.c

```
Fuerza bruta.c
1  #include <stdio.h>
2  #include <string.h>
3
4  void fuerzaBruta(char texto[], char patron[]) {
5      int n = strlen(texto);
6      int m = strlen(patron);
7      int encontrado = 0;
8
9      for (int i = 0; i < n - m + 1; i++) {
10         for (int j = 0; j < m; j++) {
11             if (texto[i + j] != patron[j]) {
12                 encontrado = 0;
13                 break;
14             }
15             if (j == m - 1) {
16                 encontrado = 1;
17             }
18         }
19         if (encontrado == 1) {
20             printf("Posicion: %d\n", i);
21             encontrado = 0;
22         }
23     }
24     printf("Cantidad de apariciones: %d\n", encontrado);
25 }
26
27 int main() {
28     char texto[100];
29     char patron[100];
30     printf("Ingrese el texto: ");
31     fgets(texto, 100, stdin);
32     printf("Ingrese el patron: ");
33     fgets(patron, 100, stdin);
34     fuerzaBruta(texto, patron);
35     return 0;
36 }
```

```
C:\Users\Ryusaki\Documents\Fuerza bruta.exe
Ingrese el texto: bulba tiene un bulbasaur en su bulba
Ingrese el patron: bulba

Posiciones donde aparece el patron:
Posicion: 0
Posicion: 15
Posicion: 31
Cantidad de apariciones: 3

-----
Process exited after 15.95 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 1: Ejemplo del algoritmo de búsqueda por fuerza bruta

En la implementación desarrollada, el texto y el patrón son almacenados mediante arreglos de caracteres. La función `fuerzaBruta()` utiliza dos ciclos para realizar las comparaciones. El ciclo externo determina las posiciones posibles de inicio del patrón, mientras que el ciclo interno compara individualmente los caracteres. Cuando el número de caracteres coincidentes alcanza la longitud total del patrón, se registra la posición correspondiente.

Este algoritmo es el más sencillo de comprender e implementar, porque no necesita estructuras auxiliares. Sin embargo, tiene como desventaja que puede realizar muchas comparaciones repetidas cuando existen coincidencias parciales.

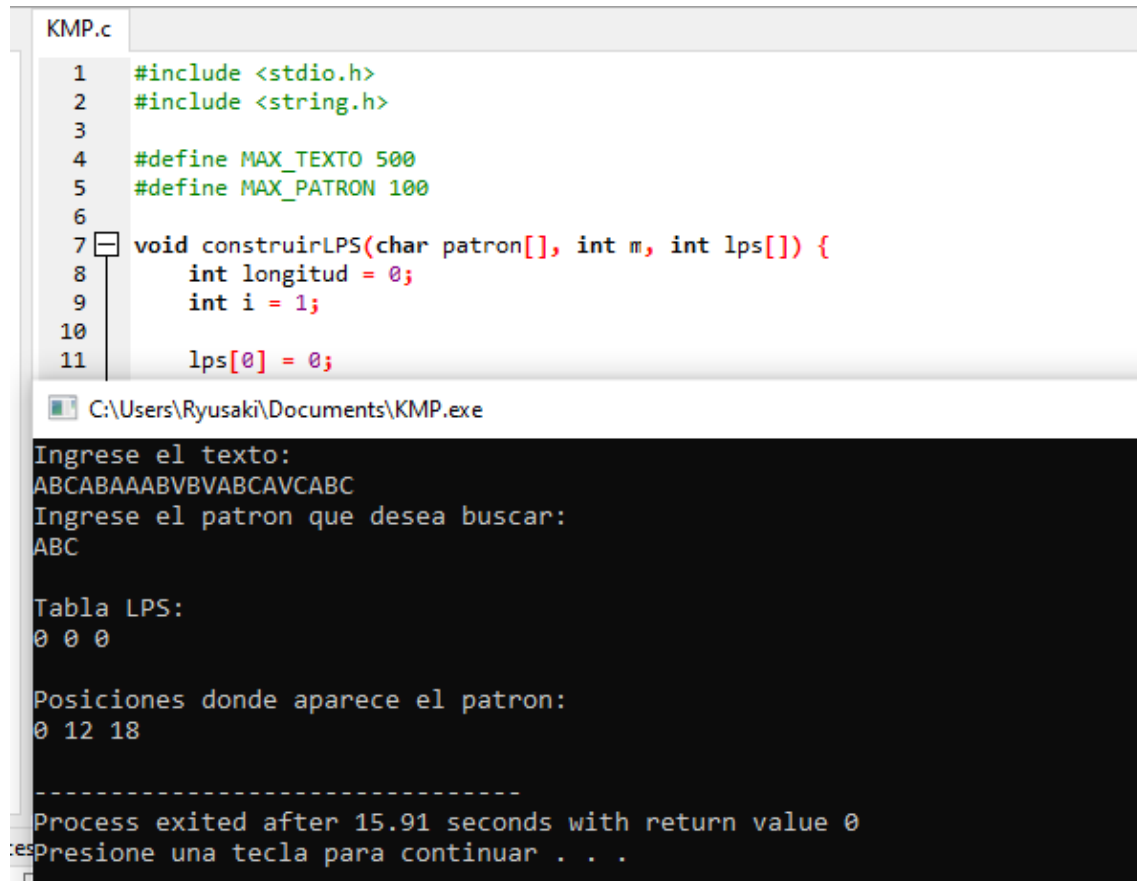
Algoritmo Knuth-Morris-Pratt (KMP)

El algoritmo Knuth-Morris-Pratt (KMP), desarrollado por Donald Knuth, James H. Morris y Vaughan Pratt, optimiza de manera radical el proceso de búsqueda al eliminar por completo la necesidad de retroceder el puntero del texto cuando ocurre un desajuste.

El principio fundamental detrás de KMP es que, al producirse un error de coincidencia, el algoritmo ya posee información valiosa sobre los caracteres que sí habían coincidido en los pasos anteriores. En lugar de descartar esta información, KMP utiliza una fase de preprocesamiento sobre el patrón para construir una tabla auxiliar (conocida comúnmente como tabla de prefijos y sufijos o función de falla).

Gracias a esta estructura, cuando se detecta un desajuste, el algoritmo consulta la tabla para determinar de forma matemática exactamente cuántas posiciones debe desplazar el patrón, garantizando que el puntero del texto jamás retroceda y logrando un rendimiento lineal estricto.

A continuación, el ejemplo del algoritmo búsqueda KMP al que le di de datos un patrón cualquiera, el código está en el archivo KMP.c



```
KMP.c
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX_TEXTO 500
5  #define MAX_PATRON 100
6
7  void construirLPS(char patron[], int m, int lps[]) {
8      int longitud = 0;
9      int i = 1;
10
11     lps[0] = 0;
```

```
C:\Users\Ryusaki\Documents\KMP.exe
Ingrese el texto:
ABCABAAABVBVABCAVCABC
Ingrese el patron que desea buscar:
ABC

Tabla LPS:
0 0 0

Posiciones donde aparece el patron:
0 12 18

-----
Process exited after 15.91 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2: Ejemplo del algoritmo de búsqueda KMP

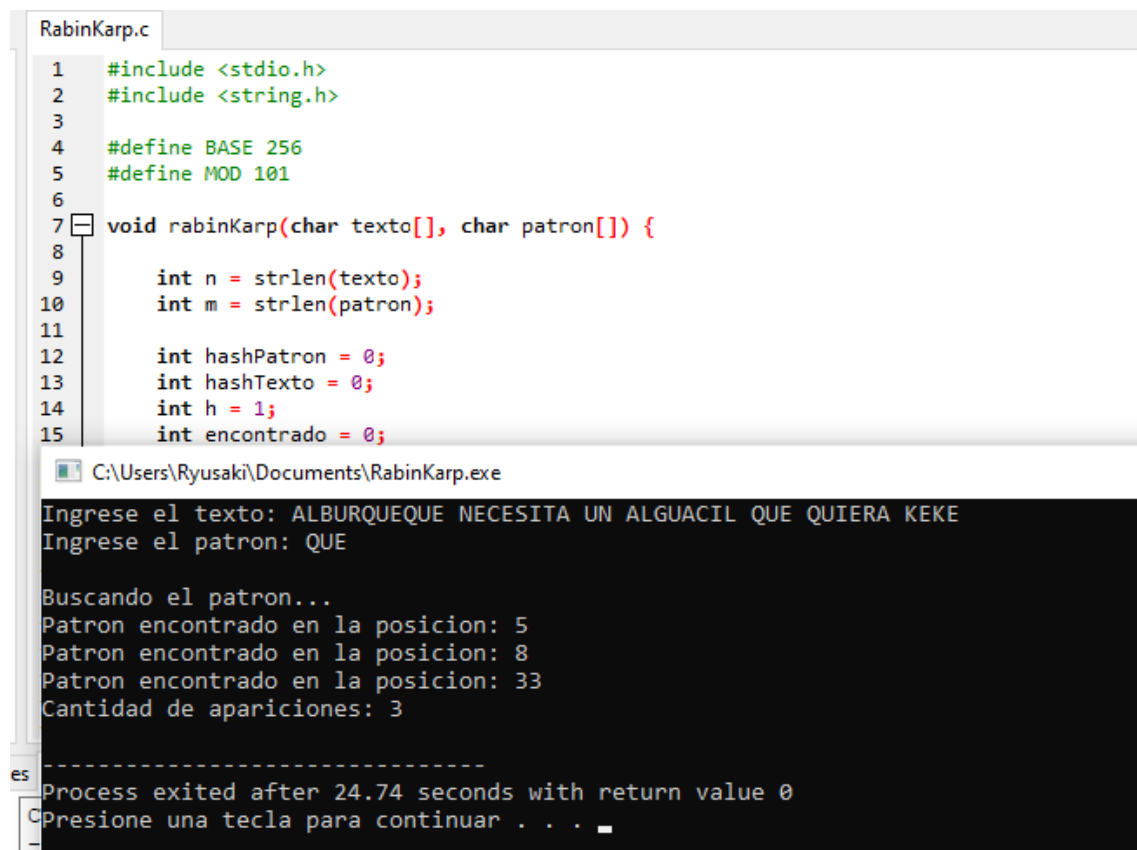
Algoritmo Rabin-Karp

Propuesto por Michael O. Rabin y Richard M. Karp, este algoritmo adopta una perspectiva completamente diferente al basar su eficiencia en el uso de funciones hash y teoría matemática de números. En lugar de comparar cadenas de caracteres de manera secuencial en cada ventana del texto, el algoritmo calcula un valor numérico único (hash) tanto para el patrón como para cada subcadena o ventana del texto que posea la misma longitud que el patrón.

Si el valor hash de la ventana del texto difiere del valor hash del patrón, se garantiza de manera inmediata que las cadenas no coinciden, por lo que el algoritmo puede avanzar a la siguiente posición sin realizar comparaciones de caracteres.

Si los valores hash coinciden (lo que indica una alta probabilidad de coincidencia), se realiza una verificación directa carácter por carácter para descartar posibles colisiones matemáticas. El aspecto más brillante de este método es la implementación de una función hash deslizante (*rolling hash*), que permite calcular el hash de la siguiente ventana en tiempo constante basándose en el resultado de la ventana anterior.

A continuación el ejemplo del algoritmo de búsqueda Rabin-Karp que se le dio datos de un patrón cualquiera



```
RabinKarp.c
1  #include <stdio.h>
2  #include <string.h>
3
4  #define BASE 256
5  #define MOD 101
6
7  void rabinKarp(char texto[], char patron[]) {
8
9      int n = strlen(texto);
10     int m = strlen(patron);
11
12     int hashPatron = 0;
13     int hashTexto = 0;
14     int h = 1;
15     int encontrado = 0;
```

```
C:\Users\Ryusaki\Documents\RabinKarp.exe
Ingrese el texto: ALBURQUEQUE NECESITA UN ALGUACIL QUE QUIERA KEKE
Ingrese el patron: QUE

Buscando el patron...
Patron encontrado en la posicion: 5
Patron encontrado en la posicion: 8
Patron encontrado en la posicion: 33
Cantidad de apariciones: 3

-----
Process exited after 24.74 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 3: Ejemplo del algoritmo de búsqueda Rabin-Karp

3. Conceptos Fundamentales: Prefijos, Sufijos y Funciones Hash

Para profundizar en la arquitectura y el funcionamiento interno de los algoritmos estudiados, resulta indispensable dominar ciertos conceptos matemáticos y formales relacionados con la manipulación de cadenas.

Prefijos y Sufijos en el Contexto del Algoritmo KMP

Dentro de la teoría de cadenas, las definiciones de prefijo y sufijo son esenciales para construir la lógica de los saltos inteligentes en KMP:

- **Prefijo:** Se define como cualquier subcadena inicial de una secuencia que comienza obligatoriamente en el primer carácter. Por ejemplo, para la cadena

"ALGORITMO", algunos de sus prefijos propios son "A", "AL", "ALGO", "ALGORIT", entre otros.

- **Sufijo:** Se define como cualquier subcadena final de una secuencia que termina obligatoriamente en el último carácter. Para la misma cadena, algunos de sus sufijos propios son "O", "MO", "ITMO", "RITMO", etcétera.

La importancia crítica de estos conceptos en el algoritmo KMP radica en el uso de la tabla auxiliar de prefijos y sufijos (tabla LPS, por sus siglas en inglés: *Longest Proper Prefix which is also Suffix*). Esta tabla almacena, para cada posición del patrón, la longitud del prefijo propio más largo que coincide con un sufijo de esa misma subcadena.

Cuando el algoritmo encuentra un desajuste en el texto, consulta esta tabla para saber qué porción del patrón ya ha sido validada y puede seguir alineada, evitando recalcular desde cero y optimizando drásticamente el tiempo de ejecución.

Función Hash y la Técnica de Hash Deslizante en Rabin-Karp

En el algoritmo Rabin-Karp, la eficiencia depende directamente del diseño de la función hash. Típicamente se emplea un enfoque de hash polinómico, donde una cadena S compuesta por m caracteres se convierte en un valor numérico mediante la siguiente sumatoria:

$$H = \sum_{i=0}^{m-1} S[i] \cdot d^{m-1-i} \pmod{q}$$

Donde d representa el tamaño del alfabeto base (por ejemplo, 256 para el código ASCII estándar) y q es un número primo suficientemente grande seleccionado para reducir la probabilidad de colisiones matemáticas y mantener las operaciones dentro de los límites de los tipos de datos estándar.

La gran ventaja computacional de este enfoque se manifiesta en la técnica de actualización mediante hash deslizante (*rolling hash*). Cuando la ventana de búsqueda avanza una posición hacia la derecha en el texto, el valor hash anterior H_{old} no necesita recalcularse por completo mediante la sumatoria. En su lugar, se actualiza en tiempo constante $O(1)$ utilizando la siguiente relación matemática:

$$H_{\text{new}} = (d \cdot (H_{\text{old}} - T[i] \cdot d^{m-1}) + T[i + m]) \pmod{q}$$

Esta fórmula resta de manera algebraica el peso del carácter que abandona la ventana por la izquierda, multiplica el resultado por la base para desplazar los pesos restantes, y finalmente suma el valor numérico del nuevo carácter que ingresa por la derecha, todo ello operando bajo el módulo q .

4. Análisis de Complejidad Computacional

Las implementaciones desarrolladas permiten observar las diferencias de complejidad entre las tres estrategias. La búsqueda por fuerza bruta realiza comparaciones independientes para cada posición posible del texto, mientras que KMP utiliza la tabla LPS para evitar comparaciones repetidas. Por su parte, Rabin-Karp utiliza valores hash para descartar rápidamente ventanas que no pueden coincidir con el patrón. Para evaluar el rendimiento de los algoritmos se utiliza la notación asintótica Big-O (O), la cual describe cómo escala el tiempo de ejecución y el consumo de memoria en función del tamaño de los datos de entrada (siendo n la longitud del texto y m la del patrón). Estos algoritmos se sitúan dentro de los problemas resolubles en tiempo polinómico (clase P), descartando complejidades exponenciales o de tipo NP. A continuación se detalla la complejidad temporal y espacial de cada enfoque:

1. Búsqueda Ingenua (Fuerza Bruta)

- **Complejidad Temporal en el Peor Caso:** $O(n \cdot m)$. Este escenario desfavorable ocurre cuando el texto y el patrón contienen secuencias altamente repetitivas (por ejemplo, un texto compuesto enteramente por la letra 'A' y un patrón compuesto por múltiples 'A' seguidas de una 'B'), lo que obliga al algoritmo a realizar comparaciones completas o casi completas en prácticamente todas las posiciones del texto.
- **Complejidad Temporal en el Mejor Caso:** $O(n)$. Se presenta cuando el primer carácter del patrón nunca coincide con los caracteres evaluados en el texto, permitiendo un desplazamiento inmediato tras una sola comparación por cada ventana.
- **Complejidad Espacial:** $O(1)$. Requiere únicamente un espacio de memoria constante y adicional mínimo, limitado a variables auxiliares para los índices de control de los ciclos.
- **Significado:** Representa el costo derivado de recorrer el texto completo y, en cada una de las $n - m + 1$ posibles posiciones de inicio, verificar potencialmente los m caracteres que componen el patrón.

2. Algoritmo Knuth-Morris-Pratt (KMP)

- **Complejidad Temporal en el Peor Caso:** $O(n + m)$. La fase inicial de preprocesamiento para la construcción de la tabla de prefijos y sufijos toma un tiempo lineal de $O(m)$, mientras que la fase de búsqueda propiamente dicha recorre el texto de manera estrictamente lineal en un tiempo de $O(n)$.
- **Complejidad Espacial:** $O(m)$. Requiere memoria adicional proporcional a la longitud del patrón para almacenar la tabla auxiliar de prefijos y sufijos (arreglo LPS).

- **Significado:** A diferencia de la fuerza bruta, el puntero del texto nunca retrocede. Cada carácter del texto es examinado un número constante de veces, lo que garantiza un rendimiento predecible, eficiente y estrictamente lineal, incluso en los escenarios más adversos.

3. Algoritmo Rabin-Karp

- **Complejidad Temporal en el Promedio:** $O(n + m)$. Siempre que la función hash distribuya de manera uniforme los valores numéricos y las colisiones sean estadísticamente excepcionales, el comportamiento del algoritmo es sumamente veloz.
- **Complejidad Temporal en el Peor Caso:** $O(n \cdot m)$. Este escenario extremo y teóricamente posible ocurre si la función hash elegida genera constantes colisiones en cada posición (por ejemplo, ataques deliberados contra la función hash), lo que obliga al algoritmo a realizar validaciones exhaustivas carácter por carácter en cada iteración.
- **Complejidad Espacial:** $O(1)$ adicional. Únicamente requiere variables numéricas auxiliares de tamaño constante para almacenar los valores hash del patrón y de la ventana deslizante actual.
- **Significado:** El cálculo inicial del hash del patrón consume un tiempo $O(m)$, y la evaluación de cada ventana sucesiva en el texto opera en tiempo constante $O(1)$ gracias a la propiedad del hash deslizante.

5. Aplicaciones y Comparación de los Algoritmos

Aplicaciones Comunes

Los algoritmos de búsqueda de patrones descritos encuentran aplicaciones directas y críticas en múltiples disciplinas tecnológicas:

- **Bioinformática:** Comparación de cadenas masivas de nucleótidos (ADN y ARN) o secuencias de aminoácidos en proteínas para identificar patrones genéticos asociados a enfermedades o características hereditarias.
- **Sistemas de Procesamiento de Textos y Editores:** Funcionalidades avanzadas de búsqueda global, análisis sintáctico y reemplazo de cadenas en entornos de desarrollo integrado (IDE) y procesadores de texto.
- **Sistemas de Detección de Intrusos (IDS) y Cortafuegos:** Inspección profunda de paquetes de red (*Deep Packet Inspection*) para reconocer firmas de código malicioso, secuencias de ataques o patrones de tráfico sospechosos en flujos de datos continuos.

- **Recuperación de Información y Motores de Búsqueda:** Indización y localización rápida de términos específicos dentro de bases de datos documentales masivas o repositorios web.

Tabla Comparativa Integral

Algoritmo	Complejidad Temporal (Peor Caso)	Complejidad Temporal (Promedio)	Complejidad Espacial	Ventajas Principales	Desventajas Principales	Situación Conveniente de Uso
Búsqueda Ingenua	$O(n \cdot m)$	$O(n \cdot m)$	$O(1)$	Máxima simplicidad conceptual; no requiere estructuras de datos auxiliares ni preprocesamiento.	Extremadamente ineficiente ante textos extensos o patrones con altas tasas de coincidencia parcial.	Textos y patrones muy cortos donde la sobrecarga de implementación de un algoritmo complejo no se justifica.
KMP	$O(n + m)$	$O(n + m)$	$O(m)$	Rendimiento lineal garantizado; el puntero del texto nunca retrocede, asegurando estabilidad temporal.	Requiere la construcción previa y el almacenamiento de una tabla auxiliar de prefijos y sufijos.	Aplicaciones generales de búsqueda de una sola cadena donde se requiera un rendimiento predecible y robusto.
Rabin-Karp	$O(n \cdot m)$	$O(n + m)$	$O(1)$	Excelente desempeño estadístico en la práctica; altamente adaptable para la búsqueda simultánea de múltiples patrones.	Vulnerable a escenarios de colisiones hash frecuentes que pueden degradar su rendimiento al nivel de fuerza	Búsqueda simultánea de múltiples patrones de la misma longitud o escenarios donde las colisiones sean estadísticamente improbables.

Figura 4: Tabla comparativa hecha en Excel

6. Conclusiones:

Conclusiones

Al realizar este trabajo pude entender mejor cómo funcionan los algoritmos de búsqueda de patrones en cadenas. Al principio pensaba que buscar una palabra dentro de un texto era algo muy sencillo y que todos los algoritmos hacían prácticamente lo mismo, pero al revisar la búsqueda por fuerza bruta, KMP y Rabin-Karp pude notar que cada uno tiene una forma diferente de resolver el mismo problema.

Una de las cosas que más me costó entender fue el algoritmo KMP, especialmente la tabla LPS, ya que anteriormente no había trabajado con este concepto. Al implementarlo pude comprender poco a poco que esta tabla sirve para saber cuánto del patrón ya coincide y así evitar volver a comparar caracteres que ya fueron revisados.

También fue nuevo para mí el concepto de hash utilizado en Rabin-Karp. Antes de realizar este trabajo no tenía mucho conocimiento sobre esta forma de comparar cadenas, por lo que al principio resultó un poco confuso. Sin embargo, al revisar el

funcionamiento del algoritmo pude entender que primero se comparan valores hash y, cuando estos coinciden, se comprueba directamente si las cadenas son iguales.

Durante la implementación en C también tuve algunas dificultades con ciertas instrucciones y librerías que no había utilizado anteriormente. Esto me permitió conocer mejor funciones para trabajar con cadenas y entender que no solamente es importante conocer la lógica del algoritmo, sino también saber cómo llevar esa lógica correctamente a un programa.

Finalmente, considero que realizar los tres algoritmos fue útil para comprender que existen diferentes maneras de solucionar un mismo problema. La búsqueda por fuerza bruta es más fácil de entender e implementar, mientras que KMP y Rabin-Karp requieren conocer conceptos adicionales, pero pueden ser más eficientes en determinadas situaciones.

7. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3.^a ed.). MIT Press.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.
- Rabin, M. O., & Karp, R. M. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2), 249-260.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley Professional.